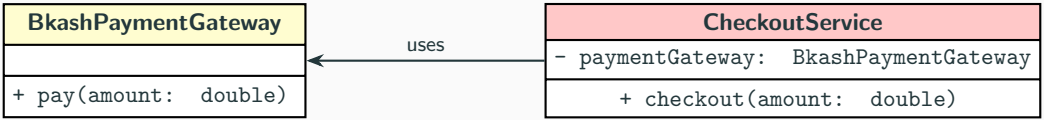
CSE214: Software Engineering Sessional

Programming to an Interface, not an Implementation

Why Tight Coupling Hurts and How Interfaces Help

The Problem: Tight Coupling

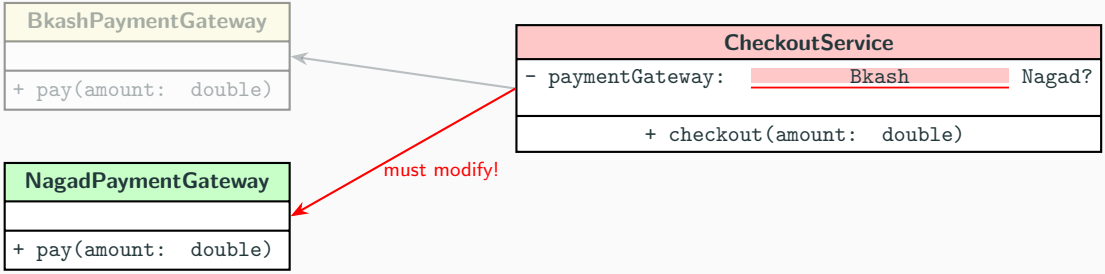


CheckoutService **directly depends** *on the concrete class BkashPaymentGateway*

Issue

Hard-coded dependency — CheckoutService **knows** exactly which payment class it uses.

What If Requirements Change?



Problems:

- Must **modify** CheckoutService source code
- Must **recompile** CheckoutService
- What if we want **both** at runtime? → if/else chaos!

The Solution: Program to an Interface

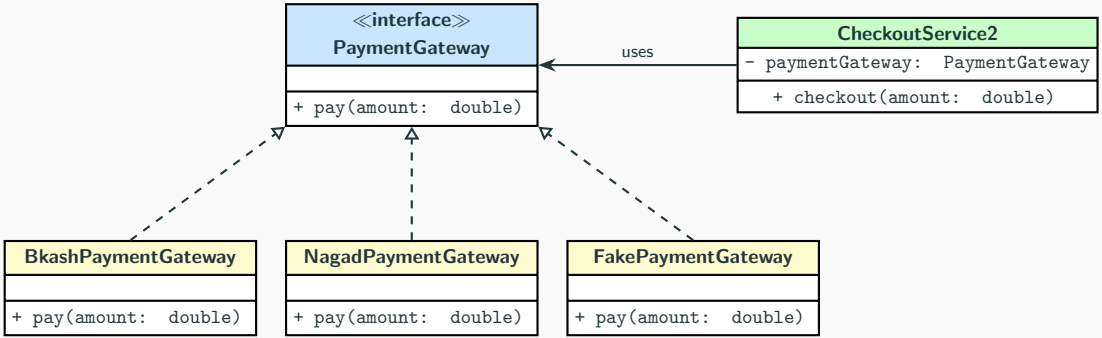
Without Interface

- Depends on **concrete** class
- Tight coupling
- Hard to change
- Hard to test

With Interface

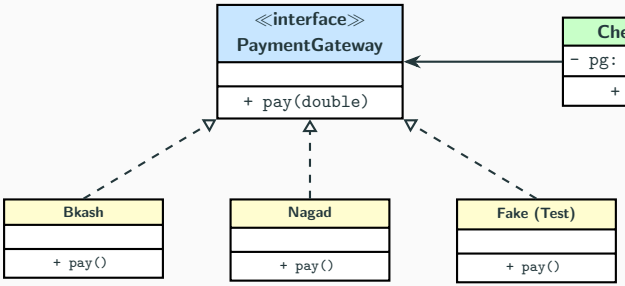
- Depends on **abstraction**
- Loose coupling
- Easy to extend
- Easy to test (mock/fake)

Design with Interface



Legend: - - - -▷ = implements ⟶ = depends on

Benefits: Flexibility at Runtime

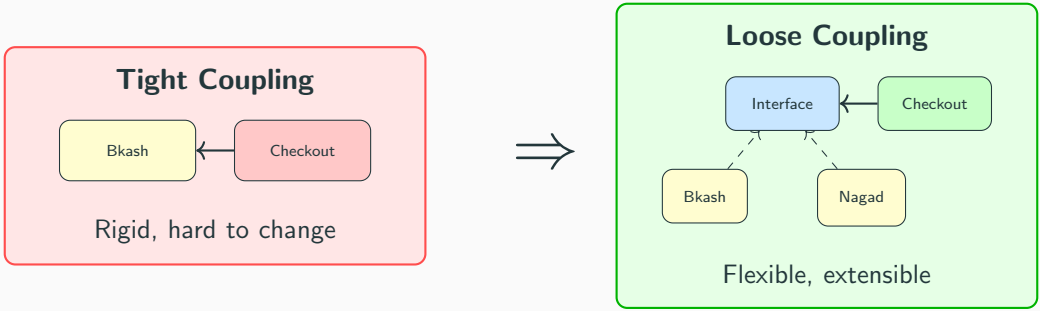


✓ **Open for Extension**
Add new gateways without modifying CheckoutService2

✓ **Closed for Modification**
CheckoutService2 code never changes

✓ **Testable**
Inject FakePaymentGateway for unit tests

Key Takeaway



Remember

"Program to an **interface**, not an **implementation**."

Now let's look at the code...